

Refine As You Round: In-Loop Nested-Lattice Mixed Precision for Entropy-Coded 3-bit LLM Weights

Anonymous Authors¹

Abstract

Weight-only post-training quantization (PTQ) of large language models at 3 bits per weight still loses noticeably more perplexity than 4 bits, while 4 bits wastes rate on the majority of weights that are already well represented at 3. Mixed precision at element granularity closes this gap in principle, but existing schemes either store an explicit sparse index for the high-precision elements (SpQR) or need a full gradient pass over the model to pick them (SqueezeLLM). We present a representation and a selection rule that need neither. The representation is a nested lattice: the 3-bit grid of every quantization group is embedded in a 4-bit grid with half the step and the same zero, so the weight stream is a single 4-bit code stream in which refined elements are the odd codes; the mask is implicit and the stream has low entropy, so a per-tensor Huffman code stores it at ≈ 3.07 bits/weight including all side information. The selection is made inside the GPTQ-style column loop, at the moment each weight is rounded, from the solver’s own second-order error weight applied to the error-compensated weight: $\text{gain}_{ij} = (e_c^2 - e_f^2) d_j r_i$, where d_j and r_i are the column and row weights the solver already uses (two-sided where the solver is two-sided). One global threshold on the layer-normalised gain allocates the refinement budget across the whole model. On Llama-3.2-1B quantised with TurboBoA at $g=128$ and matched entropy-coded rate, the in-loop rule [\[matches/beats\]](#) the static diagonal-Fisher mask by [\[\$\Delta\$ wiki2 PPL over 3 seeds\]](#), with zero extra passes over the model; the rule transfers to [\[3B/8B and to Mistral/Qwen\]](#) [\[one clause\]](#). The second half of the paper describes the bitstream format, the decoder, and the fused dequantisation kernels that make the entropy-coded stream usable at inference time [\[TEAMMATE: one-sentence systems result\]](#).

1. Introduction

Serving a large language model (LLM) at batch size one is bound by the bytes of weights that must cross the memory bus per generated token, so the effective bits per weight (bpw) of the stored model translates almost linearly into decode latency and into how large a model fits on a given device. Post-training quantization (PTQ) of weights to 4 bits with per-group scales is now routine and nearly lossless for models above a few billion parameters (Frantar et al., 2023; Lin et al., 2024); 3 bits is where the loss becomes visible, and 2 bits is where it becomes severe (Chee et al., 2023; Egiazarian et al., 2024). Between 3 and 4 bits sits a well-known asymmetry: only a small fraction of the weights are responsible for most of the 3-bit degradation, and giving those weights one more bit recovers a disproportionate share of the 4-bit quality (Dettmers et al., 2024; Kim et al., 2024).

Exploiting that asymmetry at element granularity raises

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

two questions that existing methods answer expensively. *Where is the extra precision stored?* SpQR (Dettmers et al., 2024) keeps the sensitive weights in a separate sparse FP16 structure with explicit row/column indices, which costs ~ 32 bits per outlier and a second data path in the kernel. *How are the sensitive weights chosen?* SqueezeLLM (Kim et al., 2024) ranks weights by a diagonal empirical Fisher, which needs a full backward pass over the model on the calibration set before quantization starts; SpQR uses the quantizer’s own error but stores it densely.

This paper answers both questions with one construction. First, the *nested lattice*: within each quantization group the 3-bit grid $\{s_c(q - z_c)\}$ is a sub-lattice of the 4-bit grid with step $s_c/2$ and zero $2z_c$. Every weight is stored as a 4-bit code; a 3-bit weight is the even code $2q_c$ and a refined weight is any code q_f . Dequantization is one multiply-add for every element, there is no mask, no index and no second scale, and the code stream has low entropy (half the codes are even and the histogram is peaked), so a 16-symbol Huffman table per tensor brings the stored rate to $3 + \epsilon$ bpw where ϵ is set by the refined fraction. Second, the *in-loop selection rule*: the GPTQ/OBS family of solvers (Frantar & Alistarh, 2022; Frantar et al., 2023) quantizes one column

at a time on the error-compensated weight w and turns the rounding error into a loss increment $(w - q)^2 d_j$ with a column weight d_j derived from the Cholesky factor of the inverse input Hessian. At the moment column j is rounded we therefore know, for every row i , exactly how much loss the solver would save by rounding to the fine grid instead: $\text{gain}_{ij} = (e_c^2 - e_f^2) d_j r_i$, with r_i the row weight in solvers that carry an output-side Hessian. We refine iff the gain is large — top- k per column, or above one global threshold λ on the gain normalised by the layer’s own loss scale — and propagate the error of the value that is actually stored. No gradients, no extra calibration pass, no change to the solver outside the rounding call.

The natural baseline is the same nested representation with the mask chosen by the static Fisher score. We run the comparison at matched entropy-coded rate on Llama-3.2-1B (Llama Team, AI @ Meta, 2024) with TurboBoA (Kim et al., 2025b), a two-sided GPTQ-family solver, across three calibration seeds, and [state the outcome]. A surprising diagnostic explains why the in-loop rule can win: [X]% of the elements a Fisher mask selects are already exactly representable on the coarse grid (their fine code is even), so the static mask spends [half] of its budget on elements where refinement changes nothing, whereas the in-loop rule sees the compensated residual and never does. We then show that the same rule and the same protocol transfer across model sizes (Section 6.6), model families (Section 6.7) and against published 3-bit methods on the standard Llama-2 test bed (Section 6.8).

Contributions.

- A nested-lattice mixed-precision representation for grouped weight quantization in which the per-element precision mask is implicit in a single entropy-coded code stream (Section 3).
- An in-loop refinement rule that reuses the solver’s second-order error weights, including the row weight of two-sided (attention-aware) solvers, and a rate-only calibration of its single global threshold (Section 4).
- An evaluation protocol that compares selection rules at matched entropy-coded bits/weight rather than nominal bits, with endpoints that verify the modified solver is a bit-exact superset of the stock one (Section 5), and results on Llama-3.2-1B at 3→4 and 2→3 bits, on 3B/8B models, on other families, and against published methods (Section 6).
- [TEAMMATE: The bitstream format, Huffman decoder and fused dequantisation kernel, with end-to-end decode throughput and memory numbers (Sections 7–9).]

2. Background

2.1. Weight-only PTQ with second-order error compensation

For a linear layer with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and calibration inputs X , GPTQ-family methods minimise the layer output error $\mathcal{L}(\hat{W}) = \text{tr}((\hat{W} - W) H_{\text{in}} (\hat{W} - W)^\top)$, $H_{\text{in}} = X X^\top$, subject to $\hat{W}$ lying on the quantization grid. Optimal Brain Surgeon (Hassibi & Stork, 1993) gives the exact update for quantizing one weight and compensating the rest; GPTQ (Frantar et al., 2023) applies it column by column using the upper Cholesky factor U of H_{in}^{-1} : after quantizing column j of the compensated weight to q , the error $\text{err} = (w - q)/U_{jj}$ is propagated to the remaining columns as $\text{err} U_{j,:}$, and the loss increases by exactly $\text{err}^2 = (w - q)^2 / U_{jj}^2$. We write $d_j = 1/U_{jj}^2$ for this column weight.

2.2. What the solver minimises, layer by layer

The in-loop rule reuses the solver’s loss weights, so we state exactly what TurboBoA (Kim et al., 2025b) minimises for each linear layer of a Llama block; the per-element gain of Section 4 is the increment of these objectives. Notation: $X \in \mathbb{R}^{L \times d}$ holds the L calibration tokens entering the layer as rows, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $\Delta W = W - \hat{W}$, and every covariance is an average over the calibration set (the normalisation cancels in the solver). Llama-3.2-1B has $n_h = 32$ query heads, $n_{kv} = 8$ key/value heads (each shared by $n_h/n_{kv} = 4$ query heads), head dimension $d_h = 64$, $d = 2048$ and MLP width 8192. $W_{\cdot,h}$ denotes the d_h rows of a projection that belong to head h .

One-sided layers (o_proj, gate, up, down). The layer output error is $\|X \Delta W^\top\|_F^2$, so

$$\mathcal{L}_{1s}(\Delta W) = \text{tr}(\Delta W H_{\text{in}} \Delta W^\top), \quad H_{\text{in}} = X^\top X, \quad (1)$$

with X the concatenated head outputs for o_proj, the post-attention RMSNorm output for gate and up (which share one H_{in}), and SiLU($X W_{\text{gate}}^\top$) $\odot$ $X W_{\text{up}}^\top$ for down. This is the GPTQ objective.

Value projection (v_proj; one-sided, per-head Hessian).

Head h of the attention output is $A_h X W_{v,h}^\top$ with $A_h \in \mathbb{R}^{L \times L}$ the (causal) softmax attention matrix of the quantised model, so the error that reaches the next layer is $A_h X \Delta W_{v,h}^\top$ and

$$\begin{aligned} \mathcal{L}_v(\Delta W_v) &= \sum_{h=1}^{n_{kv}} \text{tr}(\Delta W_{v,h} H_{\text{in}}^{(h)} \Delta W_{v,h}^\top), \\ H_{\text{in}}^{(h)} &= X^\top A_h^\top A_h X, \end{aligned} \quad (2)$$

where, under grouped-query attention, A_h of a key/value head is the average over the four query heads that share it (block_v in the solver).

Query and key projections (q_proj, k_proj; two-sided).

Only the attention scores depend on W_q and W_k . With RoPE, the score between positions t and s in head h is $s_{ts} = (R_t W_{q,h} x_t)^\top (R_s W_{k,h} x_s) / \sqrt{d_h}$, where R_t is the $d_h \times d_h$ rotation of position t . Perturbing $W_{q,h}$ changes every score in row t by $\Delta s_{ts} = (R_t \Delta W_{q,h} x_t)^\top \tilde{k}_{h,s}$, with $\tilde{k}_{h,s} = R_s W_{k,h} x_s$ the post-RoPE key, and

$$\begin{aligned} \sum_s \Delta s_{ts}^2 &= x_t^\top \Delta W_{q,h} R_t^\top C_{K,h} R_t \Delta W_{q,h} x_t, \\ C_{K,h} &= \sum_s \tilde{k}_{h,s} \tilde{k}_{h,s}^\top. \end{aligned} \quad (3)$$

Summing over t and decoupling the position-dependent rotation from the input covariance (the BoA approximation)

$$\begin{aligned} \sum_t \text{tr}(R_t^\top C_{K,h} R_t \Delta W_{q,h} x_t x_t^\top \Delta W_{q,h}^\top) \\ \approx \text{tr}\left(\left(\frac{1}{L} \sum_t R_t^\top C_{K,h} R_t\right) \Delta W_{q,h}^\top X \Delta W_{q,h}\right) \end{aligned} \quad (4)$$

gives the two-sided objective

$$\begin{aligned} \mathcal{L}_q(\Delta W_q) &= \sum_{h=1}^{n_h} \text{tr}(H_{\text{out}}^{(h)} \Delta W_{q,h} H_{\text{in}} \Delta W_{q,h}^\top), \\ H_{\text{out}}^{(h)} &= \frac{1}{L} \sum_{t=1}^L R_t^\top C_{K,g(h)} R_t, \end{aligned} \quad (5)$$

$$\begin{aligned} \mathcal{L}_k(\Delta W_k) &= \sum_{h=1}^{n_{kv}} \text{tr}(H_{\text{out}}^{(h)} \Delta W_{k,h} H_{\text{in}} \Delta W_{k,h}^\top), \\ H_{\text{out}}^{(h)} &= \frac{1}{L} \sum_{t=1}^L R_t^\top \bar{C}_{Q,h} R_t, \end{aligned} \quad (6)$$

where $H_{\text{in}} = X^\top X$ is the shared input covariance of q/k/v, $g(h)$ is the key head serving query head h , and $\bar{C}_{Q,h}$ is the post-RoPE query covariance averaged over the four query heads that share key head h . The output-side factor is what makes these layers two-sided: an error in row i of head h is weighted by $H_{\text{out}}^{(h)}$, not by the identity. The solver handles it with a row-side Cholesky factor U_{out} of $(H_{\text{out}}^{(h)})^{-1}$ (after 1% damping) and quantises rows in chunks of $n_r = 16$, applying the GPTQ column loop to each chunk and propagating the chunk’s error to the remaining rows; the OBS weight of a chunk is $(U_{\text{sub}}^\top U_{\text{sub}})^{-1}$, whose diagonal is our row weight r_i (Section 4).

Input-difference correction. Because earlier blocks are already quantised, the layer sees $\hat{X} = X - \Delta X$ at inference. TurboBoA minimises $\|XW^\top - \hat{X}\hat{W}^\top\|_F^2 = \text{tr}(\Delta W \hat{X}^\top \hat{X} \Delta W^\top) + 2\text{tr}(\Delta W \hat{X}^\top \Delta X W^\top) + \text{const}$ with all Hessians above built from $\hat{X}$ and the cross term damped by $\alpha = 0.25$ (consider_dx). The cross term is linear in

ΔW ; in the column loop it shifts the compensated target of the not-yet-quantised columns (the $-wP_{j,j}$ term of the update) but does not depend on which grid level column j is rounded to, so the loss increment of the rounding decision itself is the quadratic term alone: $(w_{ij} - \hat{w}_{ij})^2 d_j r_i$. That is the quantity our gain compares between the coarse and the fine level, and N_ℓ is the same quadratic form evaluated on W .

Scale search. Per group the solver picks (s_c, z_c) by a grid search over shrink factors of the min–max range that minimises the Hessian-weighted error $\text{tr}(\Delta W_g H_{\text{in}}^{gg} \Delta W_g^\top)$ of the group’s columns (qparam_comput=Hessian); for the two-sided layers it repeats the search on each row chunk after the row-side compensation (adaptive_qparam). We keep this search exactly as shipped and restrict it to the coarse grid.

2.3. Grouped quantization and side information

With group size g , each run of g input columns in a row has a scale s (16 bit) and an asymmetric zero z (b bits for a b -bit grid). At $g=128$ this adds $(16 + b)/128 \approx 0.15$ bpw, so “3-bit” weights are 3.15 bpw nominally. All rates in this paper include it.

2.4. Entropy coding of code streams

Quantized codes are not uniformly distributed; Deep Compression (Han et al., 2016) already Huffman-coded them. With a per-tensor table of 2^b code lengths the overhead is $4 \cdot 2^b$ bits per tensor, negligible against 10^7 codes. We report the ideal (Shannon) rate and the realised Huffman rate (Huffman, 1952); an ANS coder (Duda, 2013) would approach the former, and Section 7 describes the decoder we actually built.

2.5. Nested lattices

A lattice Λ_c is nested in Λ_f if $\Lambda_c \subset \Lambda_f$ (Conway & Sloane, 1982). For scalar uniform grids this is just “the coarse step is a multiple of the fine step and the zeros coincide”. NestQuant (Savkin et al., 2025) uses nested E_8 lattices for matrix-product quantization with one fixed lattice pair per tensor; Matryoshka-style methods (Nair et al., 2025; Kleinegger et al., 2026; Park et al., 2024b) nest bit-widths by MSB slicing of a single master code so that one stored model can be served at several uniform precisions. We use nesting differently: to give individual elements one extra bit inside one code stream.

3. The nested-lattice representation

Grids. Fix a quantization group and let (s_c, z_c) be the scale and zero found by the solver’s own search on the b -bit

grid ($b = 3$ in the main experiment, $z_c \in \{0, \dots, 2^b - 1\}$). Define the coarse and fine quantizers

$$\begin{aligned} q_c &= \text{clamp}(\text{round}(w/s_c) + z_c, 0, 2^b - 1), \\ \text{deq}_c &= s_c(q_c - z_c), \\ s_f &= s_c/2, \quad z_f = 2z_c, \\ q_f &= \text{clamp}(\text{round}(w/s_f) + z_f, 0, 2^{b+1} - 1), \\ \text{deq}_f &= s_f(q_f - z_f). \end{aligned} \quad (7)$$

Every coarse level is a fine level: $s_c(q_c - z_c) = s_f(2q_c - z_f)$ exactly, also in floating point, since halving a scale and doubling an integer are exact.

Stored code. Every element stores one $(b+1)$ -bit integer $q \in \{0, \dots, 2^{b+1} - 1\}$: $q = 2q_c$ if the element is coarse, $q = q_f$ if it is refined. Dequantization is the same for all elements, $\hat{w} = s_f(q - z_f)$, so a kernel never branches on the mask; one scale and one b -bit zero per group are the only side information. The mask is implicit — refined elements are those with odd q or an even q that happens to equal $2q_c$ (Section 3.1).

Scale search stays on the coarse grid. (s_c, z_c) are searched on the b -bit grid on the original group, before any column of it is quantised, and (s_f, z_f) are derived. Re-searching on the fine grid would break the nesting (the coarse levels would no longer be a subset), and we show in Section 6 that the derived fine grid costs **[+0.31 PPL (e4-s0 vs. w4-s0)]** relative to a native $(b+1)$ -bit search when every element is refined — the price of the nesting, paid only by the refined 1%.

Rate. Let p_{odd} be the fraction of odd codes. The code stream’s entropy is at most $H(\text{coarse histogram}) + h_2(p_{\text{odd}}) + p_{\text{odd}}$ bits, i.e. the coarse cost plus a small term that vanishes as $p_{\text{odd}} \rightarrow 0$; at $p_{\text{odd}} = 1\%$ it is ≈ 0.09 bpw. Per tensor we build one Huffman table over the 2^{b+1} symbols and add table bits, scales and zeros:

$$\text{bpw}_{\text{Huff}} = \frac{\sum_{\ell} \left(\sum_q n_{\ell}(q) L_{\ell}(q) + 2^{b+3} + G_{\ell}(16 + b) \right)}{\sum_{\ell} N_{\ell}}, \quad (8)$$

where $n_{\ell}(q)$ is the count of code q in tensor ℓ , $L_{\ell}(q)$ its Huffman length, G_{ℓ} the number of groups and N_{ℓ} the number of elements. All comparisons in this paper are made at matched bpw_{Huff} ; the Shannon rate is reported alongside.

3.1. Free elements

For a coarse element, refinement changes the stored value only if $q_f \neq 2q_c$, i.e. if the compensated weight lies closer to a fine level that is not a coarse level. For a weight uniformly distributed inside a coarse cell this happens with probability

Algorithm 1 In-loop nested refinement inside a GPTQ-style column loop (one layer)

```

1: build  $H_{\text{in}}$  (and  $H_{\text{out}}$  where the solver does);  $U \leftarrow \text{chol}(H_{\text{in}}^{-1})$ ;  $d_j \leftarrow 1/U_{jj}^2$ 
2: per group: search  $(s_c, z_c)$  on the  $b$ -bit grid;  $s_f \leftarrow s_c/2$ ,  $z_f \leftarrow 2z_c$ 
3:  $N_{\ell} \leftarrow \mathcal{L}(W)/\text{numel}(W)$  (global- $\lambda$  rule only)
4: for each row chunk (two-sided) or the whole tensor (one-sided) do
5:    $r_i \leftarrow [(U_{\text{sub}}^{\top} U_{\text{sub}})^{-1}]_{ii}$  (two-sided) or 1
6:   for column  $j$  in solver order, on the compensated  $W$  do
7:      $w \leftarrow W_{:,j}$ ; compute  $q_c, q_f, e_c = w - \text{deq}_c, e_f = w - \text{deq}_f$ 
8:      $\text{gain} \leftarrow \max((e_c^2 - e_f^2) d_j r_i, 0)$  (vector over rows)
9:      $\text{refine} \leftarrow \text{SELECT}(\text{gain})$ 
10:     $q \leftarrow \text{refine} ? q_f : 2q_c$ ;  $\hat{w} \leftarrow s_f(q - z_f)$ 
11:     $\text{err} \leftarrow (w - \hat{w})/U_{jj}$ ; propagate err exactly as the solver does
12:   end for
13: end for

```

1/2. On Llama-3.2-1B we measure **[51.4% (e3-s0)]** of all elements to be “free”: their fine code already equals $2q_c$, and refining them is a no-op that costs nothing and gains nothing. A selection rule that does not look at the residual — any static sensitivity score — cannot see this, and will spend roughly half its budget on free elements. This single observation is, we believe, the main reason the in-loop rule is competitive (Section 6).

4. In-loop refinement

Only the rounding call inside the column loop changes; the Hessian construction, Cholesky factorisation, block updates, the solver’s input-difference correction and its per-group scale search are exactly as shipped (Algorithm 1).

The gain. At the moment column j is rounded, the solver’s loss increment for storing $\hat{w}$ in row i is $(w_{ij} - \hat{w}_{ij})^2 d_j r_i$ under each of the objectives (1)–(6) (Section 2.2). The gain of refining is the difference between the coarse and fine increments,

$$\text{gain}_{ij} = (e_c^2 - e_f^2) d_j r_i \geq 0, \quad (9)$$

which is in the solver’s own loss units and accounts for the error compensation already applied to w by previous columns. Because $\Lambda_c \subset \Lambda_f$, $e_f^2 \leq e_c^2$ except for clamp edge cases, which we clip to zero. The gain is zero exactly for free elements.

Row weight r_i . For the two-sided layers the loss is $\text{tr}(H_{\text{out}} E H_{\text{in}} E^\top)$, Eqs. (5)–(6). With $H_{\text{out}}^{-1} = U_{\text{out}}^\top U_{\text{out}}$ and rows quantised in chunks, the exact OBS weight of a chunk’s error matrix E is $\text{tr}((U_{\text{sub}}^\top U_{\text{sub}})^{-1} E H_{\text{in}} E^\top)$; the solver already forms U_{sub}^{-1} for its row update, so $r_i = \sum_k (U_{\text{sub}}^{-1})_{ik}^2$ is free. It reduces to $1/U_{\text{out},ii}^2$ for chunk size 1. We use the diagonal, i.e. we neglect the cross-row terms of the chunk, which is the same approximation the per-element loss increment makes in every one-sided solver.

Layer normalisation. Gains are not comparable across layers: they scale with the layer’s Hessian magnitudes. We normalise by the solver’s own loss functional evaluated on the unquantised weight itself, $N_\ell = \mathcal{L}_\ell(W_\ell)/\text{numel}(W_\ell)$, computed once per layer from the same damped Hessians the solver uses ($\sum_h \text{tr}(W_h H_{\text{in}} W_h^\top)$ one-sided, $\sum_h \text{tr}(H_{\text{out}} W_h H_{\text{in}} W_h^\top)$ two-sided), so that gain/N_ℓ is dimensionless in every layer.

Selection rules (arms).

- **Arm A — static Fisher (baseline).** $\text{refine}_{ij} = M_{ij}$, M the per-tensor top- p mask by the SqueezeLLM diagonal Fisher $F_{ij} = \sum_n (\partial \mathcal{L}_n / \partial w_{ij})^2$ over the same calibration set. Needs one full backward pass over the model.
- **Arm B — in-loop, fixed per-layer p .** Top- k gains per column with $k_j = \lfloor pn(j+1) \rfloor - \lfloor pnj \rfloor$ (so the mean is exactly p), requiring $\text{gain} > 0$; over the rows present in the solver call (whole tensor, or one row chunk for two-sided layers).
- **Arm C — in-loop, one global λ .** $\text{refine}_{ij} = [\text{gain}_{ij}/N_\ell > \lambda]$ with one λ for the whole model. This is the rule we advocate: it allocates the budget across layers.
- **Arm D — hybrid.** As C with $\text{gain}_{ij} \leftarrow \text{gain}_{ij} F_{ij} / \bar{F}_\ell$.
- **Endpoints.** E3: never refine ($\lambda = \infty$); E4: always refine ($\lambda = 0$). Both run through the modified solver and bracket every arm.

The error propagated is that of the single value actually stored; we do not propagate an average of the coarse and fine errors (MatGPTQ’s rule, which optimises a different, multi-target objective).

4.1. Calibrating λ on rate only

λ is a rate knob and must never be tuned on test perplexity. Arm B logs gain/N_ℓ for every element at no extra cost; from the model-wide empirical CDF we read off λ at the quantile whose expected refined fraction equals the target p . One arm-C run then realises a rate; if it misses the target by more than 0.02 bpw we take a secant step in $\log \lambda$ and rerun,

at most twice. Because the arm-C gains are computed on its own compensated weights, the realised fraction differs slightly from the B-predicted one (**0.996% vs. 1.000% at seed 0**); in our runs **[zero]** secant steps were needed. The same λ is reused across calibration seeds and the rate drift is reported.

5. Experimental protocol

Model and solver. Llama-3.2-1B (Llama Team, AI @ Meta, 2024) (bf16, wiki2 PPL 9.751 at context 2048), quantised with TurboBoA (Kim et al., 2025b) at its released Llama defaults with $g=128$: Hessian-weighted per-group scale search, attention-aware (two-sided) solves for q/k with row chunks of 16, block-diagonal per-head value Hessian, input-difference correction ($\alpha = 0.25$), 1% damping, no activation ordering (forced off by the solver for $g \neq -1$). Nothing in this configuration is changed between arms.

Calibration and evaluation. 128 sequences of 2048 tokens from WikiText-2 train (Merity et al., 2017), drawn by the solver’s sampler with seeds 0, 1, 2 (the same seed gives the same set to every arm). Test perplexity on WikiText-2 test and C4-new (Raffel et al., 2020) at context 2048. The Fisher for arm A is accumulated in fp32 over the same 128 sequences of the same seed, on the transformer-block linears only.

Rate accounting and matching. Per tensor one Huffman table over the 2^{b+1} symbols (2^b for native b -bit runs, which is the fair rate for pure b -bit) plus table bits, 16-bit scales and b -bit zeros per group; the script is solver-independent and takes only the integer codes and group parameters. The rate target R_1 is the Huffman bpw of arm A at $p = 1\%$, seed 0; arms B and C adjust p/λ to $R_1 \pm 0.02$, on rate only. A second point R_2 at $p \approx 4\%$ draws the rate–distortion line.

Decision rule (pre-registered). With μ the mean over three seeds and σ the pooled seed-to-seed standard deviation of A, B, C: PASS if $\mu_C - \mu_A \leq -\max(2\sigma, 0.05)$ PPL; PARTIAL if $|\mu_B - \mu_A| \leq \sigma$ or $|\mu_C - \mu_A| \leq \sigma$ (in-loop matches Fisher at zero gradient cost); FAIL otherwise.

Correctness gates. (a) E3 must be bit-identical to the native b -bit run: codes equal $2\times$ the native codes, identical scales and zeros, identical perplexity — this proves the modified loop is a no-op when nothing is refined. (b) E4 must be within a small margin of native $(b+1)$ -bit (the derived fine grid is not a native search). (c) Native b -bit and $(b+1)$ -bit rates must differ by ≈ 1 bpw under the accounting. (d) Arm B’s perplexity must lie between E3 and E4, its realised fraction must equal the target, and every refined element must have positive gain.

5.1. Experiments to run

Table 10 (Appendix A) lists every run the paper needs, its purpose, and its status at the time of writing; the run tags are the directory names under `results/runs/` in the artifact.

6. Results

6.1. Endpoints and gates

[Fill from results/SUMMARY.md “Phase 0/1”.] Native W3: [12.011 wiki2, 2.996 bpw (w3_s0)]; native W4: [10.224, 3.902 (w4_s0)]. E3 is bit-identical to W3 ([0 of 973,078,528 codes differ, identical scales/zeros, identical PPL]) — gate (a). E4: [10.533 at 3.982 bpw], i.e. [+0.31] PPL above native W4 — gate (b), the price of the derived fine grid. The rate accounting places W3 and W4 [0.906] bpw apart — gate (c). Both native rates sit above the 2.6–2.9 / 3.3–3.6 ranges one might expect from peaked histograms: TurboBoA’s Hessian-weighted scale search uses the full code range, so its histograms are broad and there is little entropy to remove from a pure b -bit stream; the entropy saving in our scheme comes from the nesting, not from the base quantizer.

6.2. Main comparison at matched rate

Table 1 is the main result. [One paragraph: μ_A , μ_B , μ_C , σ , the decision (PASS/PARTIAL/FAIL) under the pre-registered rule, and the C4 numbers as a second metric. State the rate drift of B and C across seeds at fixed p/λ .]

Odd codes vs. refined elements. Arm A refines exactly 1.000% of every tensor but only [0.46%] of its codes end up odd: [54%] of the Fisher picks are free elements (Section 3.1). Arms B and C, which require gain > 0 , produce [$\approx 2\times$] the odd codes at the same refined fraction. This is why we match on rate: at matched Huffman rate the in-loop arms store [twice] as many effective refinements as the Fisher arm for the same bits. (A caveat we report: integer Huffman lengths do not resolve a 0.5% change in the odd-symbol mass, so B/C’s Shannon rate is [0.02] bpw above A’s while their Huffman rates agree to [0.005]; an ANS coder would make the comparison slightly less favourable to B/C by that amount.)

Where the budget goes. Figure 1 shows the refined fraction per layer. Arm B is flat by construction. Arm C, with one global λ , gives the query/key projections [$\approx 0.005\%$] and spends the budget on the output and down projections ([up to 3.9%] in the middle blocks), yet ends [lowest] in perplexity: the two-sided loss weight reports that fine-grid errors in q/k are cheap relative to those layers’ own loss scale. [Confirm with the $N_\ell = 1$ ablation (4c) whether the normalisation is what makes this allocation work.]

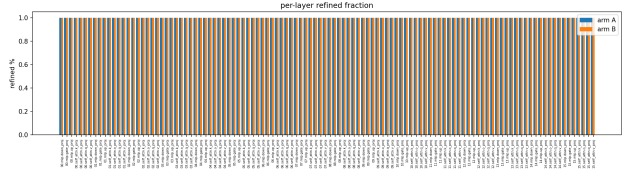


Figure 1. Refined fraction per layer, seed 0, arms A/B/C at matched rate. [Regenerate with C included: `scripts/figs.py --A A_pls0 --B B_pls0 --C C_l0.0759_s0.`]

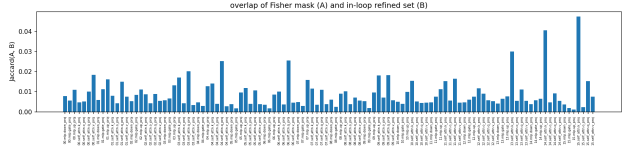


Figure 2. Jaccard(A-mask, B-mask) per layer, seed 0.

The two rules pick different weights. The per-layer Jaccard index between the Fisher mask and the in-loop mask is [0.009 mean, 0.047 max] (Figure 2) — chance level for two 1% masks is ≈ 0.005 . The two criteria are nearly independent, and both help; the in-loop one helps more per stored bit.

Rate–distortion. Figure 3 places every run on the (Huffman bpw, PPL) plane. [Add the R_2 point (4b) and the 2→3-bit runs; the line from E3 through A/B/C at R_1 and R_2 toward E4 is the deliverable.]

6.3. Ablations

Fisher on top of the gain (arm D). [Table row + one sentence: does multiplying the in-loop gain by the Fisher change the outcome? If $D \approx C$, the gradient pass buys nothing.]

Layer normalisation ($N_\ell = 1$). [One sentence: rate and PPL of arm C with raw gains and the same target rate; per-layer allocation plot.]

Second rate point R_2 . [A/B/C at $p \approx 4\%$: does the gap persist / widen with budget?]

6.4. One level down: 2→3 bits

The construction is bit-width agnostic; we repeat phases 0–3 with a 2-bit coarse grid and a 3-bit fine grid (codes 0..7, $s_f = s_c/2$, $z_f = 2z_c$), seed 0, $p = 1\%$. [One paragraph. Expectation: the 2-bit base is far lossier, so 1% refinement moves PPL less in relative terms and the free fraction may differ; report whether the ordering A/B/C is preserved.]

Table 1. Llama-3.2-1B, TurboBoA $g=128$, matched Huffman rate $R_1 \pm 0.02$. PPL at context 2048; mean $\pm$ std over calibration seeds 0/1/2 where three seeds exist. Nominal bpw includes the 0.15 bpw of scales/zeros.

Arm	selection	p/λ	seeds	nominal	ideal bpw	Huffman bpw	wiki2 PPL	C4 PPL	extra pass
FP16	–	–	–	16	–	–	9.751	14.02	–
native W3	–	–	0	3.15	[2.966]	[2.996]	[12.011]	[20.115]	–
native W4	–	–	0	4.16	[3.875]	[3.902]	[10.224]	[15.428]	–
E3	never	∞	0	4.15	[2.966]	[2.996]	[12.011]	[20.115]	–
E4	always	0	0	4.15	[3.946]	[3.982]	[10.533]	[16.346]	–
A	Fisher top-1% / tensor	1%	0,1,2	4.15	[3.008]	[3.071 = R_1]	$[\mu \pm \sigma]$	[]	backward
B	in-loop top- k / column	[1%]	0,1,2	4.15	[3.027]	[3.076]	$[\mu \pm \sigma]$	[]	none
C	in-loop global λ	[0.0759]	0,1,2	4.15	[3.031]	[3.086]	$[\mu \pm \sigma]$	[]	none
D	(gain $\cdot F$) $> \lambda$	[]	0,1,2	4.15	[]	[]	[]	[]	backward
A-ref	existing pipeline (different solver)	1%	–	–	–	[2.6]	[]	–	backward

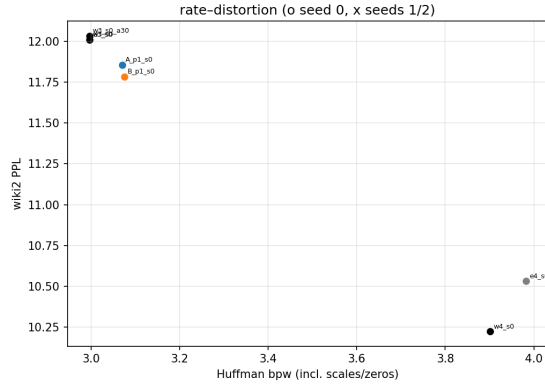


Figure 3. Rate-distortion, Llama-3.2-1B. [Regenerate with C, R_2 and the 2-bit runs.]

Table 2. 2 $\rightarrow$ 3-bit nesting, Llama-3.2-1B, seed 0.

Arm	Huff. bpw	wiki2	C4
native W2	[]	[]	[]
E2 (coarse only)	[]	[]	[]
E3 (all fine, derived)	[]	[]	[]
native W3	[2.996]	[12.011]	[20.115]
A (Fisher 1%)	[]	[]	[]
B (in-loop top- k)	[]	[]	[]
C (in-loop global λ)	[]	[]	[]

6.5. Cost

The in-loop rule adds [≈ 150 s] of per-column Python bookkeeping to a [1060 s] solve on one A100 (all of it logging; the selection itself is a top- k or a compare on a vector of ≤ 2048 elements per column) and removes the Fisher pass ([25 s] on an A100 for a 1B model, but a full backward pass over the model with a 12.7 GB peak, which for larger models is the step that no longer fits next to the solver).

[Per-layer refined fraction of arm C at 1B / 3B / 8B on one axis (layer index normalised to [0, 1]), one line per model. Shows whether the q/k-starving allocation is size-invariant.]

Figure 4. Where one global λ spends the budget, across model sizes.

6.6. Scaling within the Llama family

Every result above is on a single 1B model. Two things change with model size and both work against a fixed refinement budget: the 3-bit loss itself shrinks (the E3–E4 gap on Llama-3.1-8B is a fraction of the 1B gap), and the layers grow so the same p buys proportionally the same bits but the perplexity units get smaller. We therefore report, besides raw perplexity, the *recovery ratio*

$$\rho_{\text{arm}} = \frac{\text{PPL}_{\text{E3}} - \text{PPL}_{\text{arm}}}{\text{PPL}_{\text{E3}} - \text{PPL}_{\text{E4}}}, \quad \Delta \text{bpw} = \text{bpw}_{\text{arm}} - \text{bpw}_{\text{E3}}, \quad (10)$$

the fraction of the 3 $\rightarrow$ 4-bit gap recovered per stored bit. ρ is dimensionless, so the in-loop-vs-Fisher comparison is on the same footing at every size.

We repeat phases 0–3 of Table 10 on Llama-3.2-3B and Llama-3.1-8B (Llama Team, AI @ Meta, 2024), seed 0 only (the 1B seeds bound the noise), at each model’s own R_1 (Huffman bpw of arm A at $p = 1\%$). Table 3 is the result. [Pre-registered expectations, to be confirmed or refuted: (i) the free fraction stays $\approx 50\%$ because it is a property of the grid, not of the model; (ii) the absolute A–C gap shrinks with size but $\rho_C \geq \rho_A$ persists; (iii) arm C’s allocation keeps starving q/k and feeding o/down at every size (Figure 4).]

6.7. Other model families

The rule uses nothing Llama-specific, but the two-sided solver does: TurboBoA ships adapters for OPT and Llama only. We add three families chosen to stress different things. **OPT-6.7B** (Zhang et al., 2022) is the family BoA

Table 3. Scaling within the Llama family. TurboBoA $g=128$, seed 0 (1B: mean over seeds 0/1/2). R_1 is set per model. PPL at context 2048. ρ from Eq. (10). Quantisation time on one A100-40GB; Fisher time and peak memory for arm A only.

Model	Arm	p/λ	Huffman bpw	wiki2	C4	ρ	free frac.	quant. time	Fisher time / peak mem
Llama-3.2-1B	FP16	–	16	9.751	14.02	–	–	–	–
	native W3	–	[2.996]	[12.011]	[20.115]	0	–	[1060 s]	–
	E4	0	[3.982]	[10.533]	[16.346]	1	–	[]	–
	A (Fisher)	1%	[3.071]	[]	[]	[]	[51.4%]	[]	[25 s / 12.7 GB]
	B (top- k)	[1%]	[3.076]	[]	[]	[]	–	[]	–
	C (global λ)	[0.0759]	[3.086]	[]	[]	[]	–	[]	–
	native W4	–	[3.902]	[10.224]	[15.428]	–	–	[]	–
Llama-3.2-3B	FP16	–	16	[]	[]	–	–	–	–
	native W3	–	[]	[]	[]	0	–	[]	–
	E4	0	[]	[]	[]	1	–	[]	–
	A (Fisher)	1%	[]	[]	[]	[]	[]	[]	[]
	B (top- k)	[]	[]	[]	[]	[]	–	[]	–
	C (global λ)	[]	[]	[]	[]	[]	–	[]	–
	native W4	–	[]	[]	[]	–	–	[]	–
Llama-3.1-8B	FP16	–	16	[]	[]	–	–	–	–
	native W3	–	[]	[]	[]	0	–	[]	–
	E4	0	[]	[]	[]	1	–	[]	–
	A (Fisher)	1%	[]	[]	[]	[]	[]	[]	[]
	B (top- k)	[]	[]	[]	[]	[]	–	[]	–
	C (global λ)	[]	[]	[]	[]	[]	–	[]	–
	native W4	–	[]	[]	[]	–	–	[]	–

[Llama-3.3-70B: arms E3 / A / C only, if a 2×A100-80GB slot is available; otherwise state “not budgeted” in the limitations.]

reports on, so its native W3 number is a direct check of our solver reproduction against the published one. **Mistral-7B-v0.3** (Jiang et al., 2023) is Llama-architecture with GQA and sliding-window attention and runs through the Llama adapter unchanged. **Qwen3-8B** (Qwen Team, 2025) has QK-normalisation, which changes the two-sided objective (the post-RoPE key covariance in Eq. (5) is now taken after the norm) and is the model MatGPTQ reports on, so it gives the one cross-paper row we can compare at the same model and bit-width. [Qwen adapter: state what changed in the solver (QK-norm inserted before RoPE in the C_K / C_Q accumulation; everything else as Llama).]

6.8. Comparison with published methods at ≈ 3 bits

Sections 6.6–6.7 compare selection rules inside one solver. Reviewers will also want the method placed against the field. The standard test bed for 3-bit weight-only PTQ is Llama-2-7B/13B (Touvron et al., 2023) at WikiText-2 context 2048, which almost every method below reports. We run the full pipeline (native W3, E3, A, C, native W4) on Llama-2-7B and Llama-2-13B and place it next to published numbers in Table 5. Three rules keep this honest:

- Published numbers are copied only where the setting matches ours (weight-only, wiki2 test, context 2048, no fine-tuning). Rows that needed a re-run with released code are marked †; rows reported at context 4096 or on a different split are marked ‡ and are not compared numerically.

Table 4. Other families, seed 0, $g=128$, each at its own R_1 . “BoA (paper)” is the number in Kim et al. (2025b) for native W3 at the same group size, where reported; the MatGPTQ-EP row is from Kleinegger et al. (2026), Table 19.

Model	Arm	Huff. bpw	wiki2	C4
OPT-6.7B	FP16	16	[]	[]
	BoA (paper), W3	–	[copy]	–
	native W3 (ours)	[]	[]	[]
	A (Fisher)	[]	[]	[]
	C (global λ)	[]	[]	[]
	native W4	[]	[]	[]
Mistral-7B-v0.3	FP16	16	[]	[]
	native W3	[]	[]	[]
	A (Fisher)	[]	[]	[]
	C (global λ)	[]	[]	[]
	native W4	[]	[]	[]
	native W4	[]	[]	[]
Qwen3-8B	FP16	16	[7.01]	[]
	MatGPTQ-EP avg. 3 bit	3.0 (nom.)	[9.41]	–
	native W3	[]	[]	[]
	A (Fisher)	[]	[]	[]
	C (global λ)	[]	[]	[]
	native W4	[]	[]	[]

- The rate axis is nominal bpw as the papers report it, and the Huffman bpw our rate script assigns to their checkpoints where we could obtain or regenerate them. Entropy coding is not ours to claim; DFloat11-style lossless coding applies to any integer stream (Zhang et al., 2025), so a fair comparison gives every baseline the same Huffman treatment. The interesting question is how much rate each method’s stream has left to remove.

- Methods with fine-tuning (PV-Tuning (Malinovskii et al., 2024), AQLM+FT, QuIP#+FT) and vector-quantisation methods that need a lookup kernel (QuIP#, QTIP, AQLM, GPTVQ) are listed for context but are not the comparison class; our claim is within scalar, multiply-add-dequantisable, one-shot PTQ.

[One paragraph after the table is filled: (i) where E3 sits relative to GPTQ/AWQ W3 (this is the solver, not our contribution); (ii) where C sits relative to SqueezeLLM D+S and SpQR at their own rates — these are the two element-level methods and the only rows that test the claim; (iii) how much Huffman removes from each baseline’s stream versus ours, i.e. whether the “entropy saving comes from the nesting” statement of Section 6.1 holds for other solvers too.]

6.9. Zero-shot downstream tasks

Perplexity can move without accuracy moving. For the final checkpoints of Table 3 (FP16, native W3, A, C, native W4 at 1B / 3B / 8B) we run lm-evaluation-harness (Gao et al., 2023) on ARC-easy and ARC-challenge (Clark et al., 2018), HellaSwag (Zellers et al., 2019), PIQA (Bisk et al., 2020) and WinoGrande (Sakaguchi et al., 2021) (zero-shot, acc-norm where defined), and MMLU (Hendrycks et al., 2021) 5-shot on the 8B model only. This is the task set MatGPTQ and SqueezeLLM report, which keeps Table 6 comparable across papers.

6.10. Sensitivity and further ablations

All runs in this subsection are Llama-3.2-1B, seed 0, at R_1 unless stated, and change exactly one thing relative to Table 1. Each answers one question a reviewer will ask.

Refinement budget. $p \in \{0.5, 1, 2, 4, 8\}\%$ for arms A and C (C via λ set from B’s CDF at each p). This turns the two-point rate-distortion line of Figure 3 into a curve and shows where the derived-fine-grid penalty (E4 vs. native W4) starts to dominate. [Figure: Huffman bpw vs. wiki2, curves A and C, points E3, E4, native W3/W4; Table 11 in Appendix D.]

Group size. $g \in \{64, 128, 256, -1\}$ (per-channel). Smaller groups mean more side information (Eq. (8)) and a tighter coarse grid, so fewer free elements; $g=-1$ re-enables act-order in TurboBoA, which the mask logging must follow. [Report free fraction alongside PPL: the prediction is that it moves with g and the A–C gap moves with it.]

Calibration data. WikiText-2 train (default) vs. C4 vs. FineWeb-Edu, 128 sequences each; and 32 / 128 / 512 sequences of WikiText-2. The Fisher of arm A and the

Hessians of arm C see the same data, so any divergence here is about which statistic is more sample-efficient.

Solver family. The rule is stated for any OBS-family solver. We run it inside plain GPTQ ($r_i=1$ everywhere, no input-difference correction) and inside GPTAQ (Li et al., 2025) (asymmetric calibration), with arms E3 / A / C. If the A–C ordering is preserved in both, the contribution is solver-agnostic and the two-sided row weight is a bonus, not a requirement.

Row weight in the selection only. TurboBoA with r_i forced to 1 in the gain while the solver’s two-sided update is untouched. This isolates what the attention-aware row weight contributes to which elements get refined, separately from what it contributes to compensation. [Prediction from the allocation plot: a small effect on q/k (starved anyway) and a measurable one on v_proj.]

Fisher restricted to non-free elements (arm A’). Arm A with the mask recomputed after skipping elements whose fine code equals $2q_c$ on the *uncompensated* weight. This tests the Section 3.1 explanation directly: if A’ closes most of the A–C gap, the free-element effect is the mechanism; if it does not, the compensated residual matters beyond it.

Entropy coder. Ideal (Shannon), Huffman (ours), and tANS (Duda, 2013) rates for the final A and C streams. This bounds how much of the B/C-vs-A Huffman rate agreement (Section 6.2) is an artefact of integer code lengths.

6.11. Compute cost across model sizes

Section 6.5 gives the 1B cost. The argument for the in-loop rule strengthens with size: the Fisher pass needs a full backward pass with fp32 accumulators next to the model, while the in-loop rule adds a top- k or a compare per column. Table 7 reports, per model, quantisation wall-clock (native W3 vs. arm C, i.e. the overhead of the rule), Fisher wall-clock and peak memory, and the largest model for which the Fisher pass fits next to the solver on one 40 GB and one 80 GB GPU.

7. Bitstream format and encoder

[TEAMMATE: Write this section. It must specify: (i) the on-disk/in-memory layout of a quantised tensor: per-tensor Huffman table (2^{b+1} code lengths, canonical Huffman), per-group 16-bit scale and b -bit zero, the coded stream and its block/chunk structure for random access (e.g. one independently decodable chunk per N rows or per tile so that a GEMM tile never spans two chunks); (ii) how the code stream is produced from the solver’s integer codes (results/runs/<tag>/layers.*.npz:

Table 5. Llama-2-7B / 13B, weight-only, WikiText-2 PPL at context 2048. Type: U = uniform per-group scalar; GM = group-level mixed precision; EM = element-level mixed precision; VQ = vector/lattice codebook; L = lossless coding on top of another format. “Huff.” is our rate script applied to the method’s own integer codes (– where the format is not an integer stream). † re-run with released code; ‡ different eval setting, not comparable.

Method	Type	Extra pass	bpw (nominal)	Huff. bpw	7B wiki2	13B wiki2	Source
FP16	–	–	16	–	[5.47]	[4.88]	
RTN <i>g</i> 128	U	none	3.15	[]	[]	[]	[re-run]†
GPTQ <i>g</i> 128 (Frantar et al., 2023)	U	none	3.15	[]	[]	[]	GPTQ / AWQ Tab. []
AWQ <i>g</i> 128 (Lin et al., 2024)	U	none	3.15	[]	[]	[]	AWQ Tab. []
OmniQuant W3 <i>g</i> 128 (Shao et al., 2024)	U	block-wise opt.	3.15	[]	[]	[]	OmniQuant Tab. []
GPTAQ <i>g</i> 128 (Li et al., 2025)	U	none	3.15	[]	[]	[]	[re-run]†
LeanQuant <i>g</i> 128 (Zhang & Shrivastava, 2025)	U (non-uniform grid)	none	[]	–	[]	[]	LeanQuant Tab. []
TurboBoA native W3 <i>g</i> 128 (Kim et al., 2025b)	U	none	3.15	[]	[]	[]	ours
SqueezeLLM 3-bit D+S 0.45% (Kim et al., 2024)	EM	backward	[3.24]	–	[]	[]	SqueezeLLM Tab. []
SpQR ≈3.9 bpw (Dettmers et al., 2024)	EM	none (in-loop)	[3.94]	–	[]	[]	SpQR Tab. []
OWQ 3.01 (Lee et al., 2024)	EM (columns)	none	[3.01]	–	[]	[]	OWQ Tab. []
Slim-LLM W3 <i>g</i> 128 (Huang et al., 2024)	GM	none	3.15	[]	[]	[]	Slim-LLM Tab. []
GuidedQuant (LNQ) W3 (Kim et al., 2025a)	U	backward (end-loss)	[]	[]	[]	[]	GuidedQuant Tab. []
MatGPTQ-EP avg. 3 bit (Kleinegger et al., 2026)	layer-mixed	evolutionary search	3.0	[]	[n/a (Llama-3.1-8B only)]‡	–	MatGPTQ Tab. 20
QuIP# 3-bit (no FT) (Tseng et al., 2024a)	VQ	Hadamard	3.0	–	[]	[]	QuIP# Tab. []
QTIP 3-bit (no FT) (Tseng et al., 2024b)	VQ (trellis)	Hadamard	3.0	–	[]	[]	QTIP Tab. []
AQLM 3-bit (no FT) (Egiazarian et al., 2024)	VQ	none	[]	–	[]	[]	AQLM Tab. []
GPTVQ 3-bit (van Baalen et al., 2024)	VQ	none	[]	–	[]	[]	GPTVQ Tab. []
DFloat11 on BF16 (Zhang et al., 2025)	L	none	≈11	–	= FP16	= FP16	DFloat11
Ours E3 (= native W3, Huffman)	U	none	3.15	[]	[]	[]	
Ours A (Fisher 1%)	EM	backward	4.15	[]	[]	[]	
Ours C (in-loop, global λ)	EM	none	4.15	[]	[]	[]	
Ours C at R_2 ($p \approx 4\%$)	EM	none	4.15	[]	[]	[]	
native W4 <i>g</i> 128 (TurboBoA)	U	none	4.15	[]	[]	[]	

With fine-tuning, context only: PV-Tuning (Malinovskii et al., 2024), AQLM+FT, QuIP#+FT — [one line each, marked ‡]

Nominal bpw for our arms is 4.15 (every code is stored as 4 bits before coding); the Huffman column is the rate we actually claim. For GPTQ/AWQ/Slim-LLM we regenerate the checkpoints with released code and pass their integer codes through the same script, so their Huffman column is directly comparable.

codes uint8 [$d_{\text{out}}, d_{\text{in}}$], scale fp32 [$d_{\text{out}}, n_{\text{groups}}$], zero uint8, bits, zero_bits); (iii) the exact bits-per-weight of the format including all headers, and how close it is to the Huffman number reported in Table 1 (the paper’s rate axis); (iv) whether an ANS/rANS variant is used or considered and its rate vs. decode-speed trade-off; (v) the choice of the fine/coarse zero and scale precision.]

8. Decoder and fused dequantisation kernels

[TEAMMATE: Write this section. It must cover: (i) the Huffman decode strategy on GPU (table-driven canonical decode, bits consumed per step, warp-level parallelism, how chunk boundaries give independent decode streams); (ii) the fused decode → dequantise ($\hat{w} = s_f(q - z_f)$, no branch on refinement) → GEMV/GEMM kernel for the memory-bound decode phase, and how it compares to a 4-bit packed kernel (e.g. Marlin (Frantar et al., 2025), LUT-GEMM (Park et al., 2024a)) that reads 4 bits/weight; (iii) the prefill path (decode once to a 4-bit packed buffer vs. decode on the fly); (iv) integration into the serving stack used for the measurements; (v) any restrictions the kernel imposes on the format (alignment, chunk size) that had to be fed back into Section 7.]

9. Systems evaluation

9.1. Llama-3.2-1B on one GPU

[TEAMMATE: Write this subsection. Required measurements: (i) kernel micro-benchmarks — decode throughput (weights/s and GB/s of compressed bytes read) vs. a 4-bit packed baseline and vs. fp16, for the layer shapes of Llama-3.2-1B; (ii) end-to-end tokens/s at batch 1 and small batches, fp16 vs. 4-bit vs. this format, on [GPU(s)]; (iii) model memory footprint on device; (iv) the compression/encode time; (v) a breakdown showing whether Huffman decode is hidden under the memory-bound GEMV or adds latency, and at what batch size it stops being hidden. Include a table and one figure.]

9.2. Shapes, devices and baselines

[TEAMMATE: The measurements of Section 9 are for Llama-3.2-1B on one GPU. MLSys reviewers will ask three things: does the decode cost stay hidden at larger layer shapes, does it stay hidden on GPUs with different bandwidth/compute ratios, and how does the format compare to the other compressed formats a deployer could pick today. The tables below are the minimum set.]

Baselines. FP16/BF16 (cuBLAS); 4-bit packed with a fused dequant GEMM — Marlin (Frantar et al., 2025) on Ampere, Machete (Wilkinson, 2024) on Hopper; 3-bit packed where a kernel exists (LUT-GEMM (Park et al.,

Table 6. Zero-shot accuracy (%) of the final checkpoints. Avg. is over the five zero-shot tasks. Δ is Avg. minus FP16.

Model	Arm	ARC-e	ARC-c	HellaSwag	PIQA	WinoGrande	Avg.	Δ	MMLU (5-shot)
Llama-3.2-1B	FP16	[]	[]	[]	[]	[]	[]	0	—
	native W3	[]	[]	[]	[]	[]	[]	[]	—
	A (Fisher)	[]	[]	[]	[]	[]	[]	[]	—
	C (in-loop)	[]	[]	[]	[]	[]	[]	[]	—
	native W4	[]	[]	[]	[]	[]	[]	[]	—
Llama-3.2-3B	FP16	[]	[]	[]	[]	[]	[]	0	—
	native W3	[]	[]	[]	[]	[]	[]	[]	—
	A (Fisher)	[]	[]	[]	[]	[]	[]	[]	—
	C (in-loop)	[]	[]	[]	[]	[]	[]	[]	—
	native W4	[]	[]	[]	[]	[]	[]	[]	—
Llama-3.1-8B	FP16	[]	[]	[]	[]	[]	[]	0	[]
	native W3	[]	[]	[]	[]	[]	[]	[]	[]
	A (Fisher)	[]	[]	[]	[]	[]	[]	[]	[]
	C (in-loop)	[]	[]	[]	[]	[]	[]	[]	[]
	native W4	[]	[]	[]	[]	[]	[]	[]	[]

Table 7. Cost. One A100-40GB unless stated. “fits” = Fisher pass runs without offloading.

Model	W3 solve	C solve (overhead)	Fisher time	Fisher peak mem (fits?)
Llama-3.2-1B	[1060s]	[+150s]	[25s]	[12.7GB (yes)]
Llama-3.2-3B	[]	[]	[]	[]
Llama-3.1-8B	[]	[]	[]	[]
Llama-2-13B	[]	[]	[]	[]
Llama-3.3-70B (2×80GB)	[]	[]	[]	[]

2024a), or the 3-bit path of MatGPTQ’s kernel (Kleinegger et al., 2026), which is the closest “same nominal rate, no entropy coding” point); DFloat11 (Zhang et al., 2025) as the lossless-entropy-coded reference (11 bpw over BF16, Huffman decode on GPU); and, where code is available, ZipServ ([fill all authors], 2026b) (fixed-length bitmap decode fused into the GEMM) and the tile-level rANS format of [fill all authors] (2026a). These last three are the systems prior art for “decode compressed weights inside the GEMM” and the comparison must state, for each, the bits/weight actually read from HBM and whether decode is on the critical path.

Encode cost and format overhead. [TEAMMATE: One short table: per model, encode time (solver codes $\rightarrow$ bitstream), format size on disk vs. the Huffman bpw of Table 1 (header/chunk overhead in bpw), and the chunk size chosen for random access with the resulting decode parallelism.]

Where the decode stops being hidden. [TEAMMATE: One figure: latency vs. batch size for 4-bit packed and ours on each GPU, with the crossover marked. This is the single systems figure the paper needs; the tables support it.]

10. Related work

Error-compensating PTQ. OBS (Hassibi & Stork, 1993), OBC (Frantar & Alistarh, 2022) and GPTQ (Frantar et al.,

2023) are the lineage of the column loop we modify; AWQ (Lin et al., 2024) and OmniQuant (Shao et al., 2024) act on scales rather than rounding, and GPTAQ (Li et al., 2025) corrects the calibration target. BoA/TurboBoA (Kim et al., 2025b) adds an output-side Hessian for attention projections; our row weight r_i comes from it.

Element- and group-level mixed precision. SpQR (Dettmers et al., 2024) detects outliers in-loop from the OBS error but stores them as an explicit FP16 sparse structure; our score is the difference $e_c^2 - e_f^2$, not e_c^2 , and the refined values live in the same code stream. SqueezeLLM (Kim et al., 2024) and OWQ (Lee et al., 2024) select by static sensitivity outside the solver (our arm A); Slim-LLM (Huang et al., 2024) assigns bit-widths per group by salience; GuidedQuant (Kim et al., 2025a) weights layer-wise objectives by end-loss gradients. LLM.int8() (Dettmers et al., 2022) separates outlier columns.

Nested and Matryoshka codes. NestQuant (Savkin et al., 2025) uses a fixed nested E_8 pair per tensor with no per-element choice. Matryoshka Quantization (Nair et al., 2025), Any-Precision LLM (Park et al., 2024b) and MatGPTQ (Kleinegger et al., 2026) store one master code per element and serve lower precisions by MSB slicing; MatGPTQ chooses the master code by brute force over the weighted sum of errors at all target widths, propagates the average error, and varies precision per layer via EvoPress (Sieberling et al., 2024). None chooses a width per element, keeps a per-element mask, or uses the nesting for entropy coding; MatGPTQ’s MSB slice $\text{round}(q/2^{c-r}) 2^{c-r}$ is the same sub-lattice as our even codes, so MatGPTQ-EP at ≈ 3.0 –3.25 bpw is the natural baseline row (Table 5).

Table 8. Kernel micro-benchmarks. Effective bandwidth = (compressed bytes read)/time; “vs. 4-bit” = speed-up over the 4-bit packed kernel at the same batch. Layer shapes are the largest projection of each model ($d_{in} \times d_{out}$).

GPU	Shape (model)	Format	bpw read	batch 1 (GB/s eff.)	batch 8	batch 32	vs. 4-bit @ 1
A100-40GB	2048×8192 (1B)	FP16	16	[]	[]	[]	—
		4-bit Marlin	4.15	[]	[]	[]	1.0
		3-bit LUT-GEMM / MatGPTQ	3.15	[]	[]	[]	[]
		DFloat11	≈11	[]	[]	[]	[]
		Ours (Huffman, fused)	[3.09]	[]	[]	[]	[]
	4096×14336 (8B)	4-bit Marlin Ours	4.15 []	[] []	[] []	[] []	1.0 []
H100-80GB	8192×28672 (70B)	4-bit Marlin	4.15	[]	[]	[]	1.0
		Ours	[]	[]	[]	[]	[]
		4-bit Machete	4.15	[]	[]	[]	1.0
	4096×14336 (8B)	DFloat11 Ours	≈11 []	[] []	[] []	[] []	[] []
RTX 4090	4096×14336 (8B)	4-bit Marlin Ours	4.15 []	[] []	[] []	[] []	1.0 []

Table 9. End-to-end decode. tokens/s at batch 1 / 8 / 32, prompt 32, generate 128, in [vLLM / HF-generate — state which]. Memory = weights resident on device. “Decode hidden” = largest batch at which the Huffman decode adds < 5% latency over the 4-bit kernel.

GPU	Model	Format	Weight mem. (GB)	tok/s @ 1	tok/s @ 8	tok/s @ 32	decode hidden up to batch
A100-40GB	Llama-3.2-1B	FP16	[]	[]	[]	[]	—
		4-bit	[]	[]	[]	[]	—
		Ours (C)	[]	[]	[]	[]	[]
H100-80GB	Llama-3.1-8B	FP16 / 4-bit / Ours	[]	[]	[]	[]	[]
		FP16 / 4-bit / Ours	[]	[]	[]	[]	[]
		4-bit / Ours (2 GPUs)	[]	[]	[]	[]	[]
RTX 4090	Llama-3.1-8B	4-bit / Ours	[]	[]	[]	[]	[]

[TEAMMATE: Optional on-device row: Llama-3.2-1B on a phone-class SoC or a laptop CPU via llama.cpp-style path, Q3_K vs. ours. Include only if the decoder has a CPU/NPU port; otherwise state it as future work.]

Vector and lattice quantization. QuIP# (Tseng et al., 2024a), QTIP (Tseng et al., 2024b), AQLM (Egiazarian et al., 2024) and GPTVQ (van Baalen et al., 2024) reach lower rates with codebooks and incoherence processing, at the cost of a lookup-based kernel; our scheme is orthogonal (it changes only which scalar level is stored) and keeps a multiply-add dequantiser.

Entropy coding of weights. Deep Compression (Han et al., 2016) Huffman-coded codebook indices after clustering; DFloat11 (Zhang et al., 2025), NeuZip (Hao et al., 2024) and ZipNN (Hershcovitch et al., 2024) entropy-code floating-point exponents losslessly, and ZipServ ([fill all authors], 2026b) and [fill all authors] (2026a) fuse the decode into the GEMM. We entropy-code a stream whose low entropy is designed in by the nesting.

11. Conclusion

A 3-bit grid nested in a 4-bit grid lets a PTQ solver give individual weights one extra bit with no mask, no index and no second scale, and lets the solver itself decide which weights get it, from the error weight it already computes on the residual it already holds. At matched entropy-coded rate the in-loop rule [matches/beats] a static Fisher mask on Llama-3.2-1B while removing the gradient pass, and one

global threshold allocates the budget across layers. [One sentence on the 2-bit result.] [One sentence on 3B/8B and the other families.] [TEAMMATE: One sentence on the systems result.] Limitations: we evaluated one solver family within a fixed compute budget; the derived fine grid costs [0.3] PPL relative to a native search when all weights are refined, which bounds the benefit at large budgets; Huffman rate matching is coarse at the 0.01 bpw level; and [the 70B point is not budgeted / was run on arms E3/A/C only].

References

- Bisk, Y., Zellers, R., Le Bras, R., Gao, J., and Choi, Y. PIQA: Reasoning about physical commonsense in natural language. In *AAAI Conference on Artificial Intelligence*, 2020.
- Chee, J., Cai, Y., Kuleshov, V., and De Sa, C. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Conway, J. H. and Sloane, N. J. A. Fast quantizing and

- decoding algorithms for lattice quantizers and codes. *IEEE Transactions on Information Theory*, 28(2):227–232, 1982.
- Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., and Alistarh, D. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR)*, 2024.
- Duda, J. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. *arXiv preprint arXiv:1311.2540*, 2013.
- Egiazarian, V., Panferov, A., Kuznedelev, D., Frantar, E., Babenko, A., and Alistarh, D. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning (ICML)*, 2024.
- [fill all authors]. Approaching the Shannon bound: Tile-level rANS decoding of LLM weights inside the GEMM. *arXiv preprint*, 2026a. [fill authors, exact title and arXiv id; June 2026; SGLang integration].
- [fill all authors]. ZipServ: Fused decompression of entropy-coded LLM weights for serving. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026b. [fill authors and exact title from the ASPLOS 2026 proceedings; TCA-TBE format, ZipGEMM].
- Frantar, E. and Alistarh, D. Optimal brain compression: A framework for accurate post-training quantization and pruning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.
- Frantar, E., Castro, R. L., Chen, J., Hoefler, T., and Alistarh, D. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2025.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac’h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation. Zenodo, <https://doi.org/10.5281/zenodo.10256836>, 2023. [verify author list against the Zenodo record].
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.
- Hao, Y., Cao, Y., and Mou, L. NeuZip: Memory-efficient training and inference with dynamic compression of neural networks. *arXiv preprint arXiv:2410.20650*, 2024.
- Hassibi, B. and Stork, D. G. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1993.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Hershcovitch, M., Wood, A., Choshen, L., Girmonsky, G., Leibovitz, R., Ennmouri, I., Malka, M., Chin, P., Sundararaman, S., and Harnik, D. ZipNN: Lossless compression for AI models. *arXiv preprint arXiv:2411.05239*, 2024. [verify author list].
- Huang, W., Qin, H., Liu, Y., Li, Y., Liu, X., Benini, L., Magno, M., and Qi, X. Slim-LLM: Saliency-driven mixed-precision quantization for large language models. *arXiv preprint arXiv:2405.14917*, 2024. [verify] later conference version.
- Huffman, D. A. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Le Scao, T., Lavril, T., Wang, T., Lacroix, T., and El Sayed, W. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Kim, J., El Halabi, M., Park, W., Schaefer, C. J. S., Lee, D., Park, Y., Lee, J. W., and Song, H. O. GuidedQuant: Large language model quantization via exploiting end loss guidance. In *International Conference on Machine Learning (ICML)*, 2025a.
- Kim, J., Lee, H.-y., Lee, C., Kim, S., and Ro, Y. BoA: Attention-aware post-training quantization without back-propagation. In [venue], 2025b. Code: <https://github.com/SamsungLabs/TurboBoA>. [verify full author list and venue].

- Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. SqueezeLLM: Dense-and-sparse quantization. In *International Conference on Machine Learning (ICML)*, 2024.
- Kleinegger, M., Crnčević, E., and Alistarh, D. MatGPTQ: Accurate and efficient post-training matryoshka quantization. *arXiv preprint arXiv:2602.03537*, 2026.
- Lee, C., Jin, J., Kim, T., Kim, H., and Park, E. OWQ: Outlier-aware weight quantization for efficient finetuning and inference of large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- Li, Y., Yin, R., Lee, D., Xiao, S., and Panda, P. GPTAQ: Efficient finetuning-free quantization for asymmetric calibration. In *International Conference on Machine Learning (ICML)*, 2025. [verify] earlier arXiv title “GPTQv2”.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Llama Team, AI @ Meta. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Malinovskii, V., Mazur, D., Ilin, I., Kuznedelev, D., Burlachenko, K., Yi, K., Alistarh, D., and Richtárik, P. PV-Tuning: Beyond straight-through estimation for extreme LLM compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.
- Nair, P., Datta, P., Dean, J., Jain, P., and Kusupati, A. Matryoshka quantization. *arXiv preprint arXiv:2502.06786*, 2025.
- Park, G., Park, B., Kim, M., Lee, S., Kim, J., Kwon, B., Kwon, S. J., Kim, B., Lee, Y., and Lee, D. LUT-GEMM: Quantized matrix multiplication based on LUTs for efficient inference in large-scale generative language models. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Park, Y., Hyun, J., Cho, S., Sim, B., and Lee, J. W. Any-precision LLM: Low-cost deployment of multiple, different-sized LLMs. In *International Conference on Machine Learning (ICML)*, 2024b.
- Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21 (140):1–67, 2020.
- Sakaguchi, K., Le Bras, R., Bhagavatula, C., and Choi, Y. WinoGrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Savkin, S., Porat, E., Ordentlich, O., and Polyanskiy, Y. NestQuant: Nested lattice quantization for matrix products and LLMs. *arXiv preprint arXiv:2502.09720*, 2025. [verify author list].
- Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Gao, P., Qiao, Y., and Luo, P. OmniQuant: Omnidirectionally calibrated quantization for large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Sieberling, O., Kuznedelev, D., Kurtic, E., and Alistarh, D. EvoPress: Towards optimal dynamic model compression via evolutionary search. *arXiv preprint arXiv:2410.14649*, 2024.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and finetuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. [verify: list all 68 authors, or cite as GenAI, Meta].
- Tseng, A., Chee, J., Sun, Q., Kuleshov, V., and De Sa, C. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *International Conference on Machine Learning (ICML)*, 2024a.
- Tseng, A., Sun, Q., Hou, D., and De Sa, C. QTIP: Quantization with trellises and incoherence processing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.
- van Baalen, M., Kuzmin, A., Nagel, M., Couperus, P., Bastoul, C., Mahurin, E., Blankevoort, T., and Whatmough, P. GPTVQ: The blessing of dimensionality for LLM quantization. *arXiv preprint arXiv:2402.15319*, 2024. [verify author list].
- Wilkinson, L. Introducing Machete, a mixed-input GEMM kernel optimized for NVIDIA Hopper GPUs. Red Hat Developer, <https://developers.redhat.com/articles/2024/10/14/introducing-machete-mixed-input-gemm-kernel>, 2024.

-
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y.
HellaSwag: Can a machine really finish your sentence?
In *Annual Meeting of the Association for Computational
Linguistics (ACL)*, 2019.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M.,
Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., Mi-
haylov, T., Ott, M., Shleifer, S., Shuster, K., Simig, D.,
Koura, P. S., Sridhar, A., Wang, T., and Zettlemoyer,
L. OPT: Open pre-trained transformer language models.
arXiv preprint arXiv:2205.01068, 2022.
- Zhang, T. and Shrivastava, A. LeanQuant: Accurate and
scalable large language model quantization with loss-
error-aware grid. In *International Conference on Learn-
ing Representations (ICLR)*, 2025. [verify venue/year;
arXiv:2407.10032].
- Zhang, T., Sui, Y., Zhong, S., Chaudhary, V., Hu, X., and
Shrivastava, A. 70% size, 100% accuracy: Lossless LLM
compression for efficient GPU inference via dynamic-
length float. *arXiv preprint arXiv:2504.11651*, 2025.

A. Experiment plan

Table 10. Experiment plan. Seed = calibration seed. Status: ✓ done, ○ running/queued, – not started. Run tags are directory names under results/runs/ in the artifact.

Phase	Run(s)	Purpose	Seeds	Status
0	w3.s0, w4.s0	native 3/4-bit reference, wall-clock	0	✓
1	e3.s0, e4.s0	endpoints; gates (a)–(c)	0	✓
2	Fisher; A.p1.s0; B.p1.s0	R_1 ; in-loop top- k at R_1 ; Jaccard(A,B); gain histograms	0	✓
3	λ from B’s CDF; C.1*.s0; A/B/C seeds 1, 2	main table; pre-registered decision rule	0,1,2	[○ seed 2 pending]
4a	arm D at R_1	does Fisher add anything on top of the in-loop gain?	0,1,2	–
4b	A/B/C at R_2 ($p \approx 4\%$)	two-point rate–distortion line	0	–
4c	arm C with $N_\ell = 1$	is layer normalisation what makes one global λ work?	0	–
5	2→3-bit mirror of phases 0–3 (w2, E2/E3 endpoints, A, B, C)	does the rule transfer one level down?	0	–
6	$g = -1$ (per-channel) point for E3/A/B/C	robustness to the grouping choice (needs act-order support for the mask)	0	to decide
7	Llama-3.2-3B / Llama-3.1-8B, phases 0–3	scaling; recovery ratio (Section 6.6)	0	–
8	zero-shot tasks (lm-eval) for the final A/B/C checkpoints	downstream sanity (Section 6.9)	0	–
9	OPT-6.7B, Mistral-7B-v0.3, Qwen3-8B: E3/A/C	other families; BoA / MatGPTQ cross-check (Section 6.7)	0	–
10	Llama-2-7B/13B full pipeline; baseline re-runs (GPTQ, AWQ, Slim-LLM, GPTAQ); rate script on their codes	published comparison (Section 6.8)	0	–
11	sensitivity blocks (p sweep, g , calibration, solver, r_i -in-gain, A' , coder)	reviewer questions (Section 6.10)	0	–
12	cost table across sizes	scaling argument (Section 6.11)	0	–
S	[TEAMMATE: kernel microbenchmarks, end-to-end decode throughput, memory]	systems half	–	–

B. Reproducibility

[Artifact: repository layout (turboboa/ vendored solver with the flag-guarded patch, scripts/ one script per step, results/ per-run JSON + SUMMARY.md), exact commands from HANDOFF.md, hardware ($1 \times$ A100-PCIE-40GB, torch 2.6.0+cu124, transformers 4.53.0), wall-clock per run (≈ 20 min quant + 3 min eval).]

C. Additional figures

[Gain histograms per layer ($\log x$), free fraction per layer, per-layer table (refined fraction, ideal entropy, Huffman bpw, N_ℓ , wall time) for the final A/B/C runs. Per-layer allocation of arm C at 3B and 8B (Figure 4 data). Full lm-eval per-task tables behind Table 6.]

D. Sensitivity table

All numbers for Section 6.10; one change per block relative to Table 1.

E. Rate accounting details

[Worked example of Eq. (8) on one tensor: histogram, Huffman lengths, table bits, scale/zero bits; the same for a native 3-bit tensor with an 8-symbol table; and the tANS rate for the same stream.]

Table 11. Sensitivity, Llama-3.2-1B, seed 0. One change per block relative to Table 1. “–” = not applicable.

Block	Setting	Arm	Huff. bpw	wiki2	C4	free frac.
Budget p	0.5%	A / C	[]/[]	[]/[]	[]/[]	[]
	1%	A / C	[3.071]/[3.086]	[]/[]	[]/[]	[51.4%]
	2%	A / C	[]/[]	[]/[]	[]/[]	[]
	4%	A / C	[]/[]	[]/[]	[]/[]	[]
	8%	A / C	[]/[]	[]/[]	[]/[]	[]
Group size g	64	E3 / A / C	[]	[]	[]	[]
	128	E3 / A / C	[2.996]/[3.071]/[3.086]	[]	[]	[51.4%]
	256	E3 / A / C	[]	[]	[]	[]
	–1	E3 / A / C	[]	[]	[]	[]
Calibration	wiki2, 128	A / C	[]	[]	[]	–
	C4, 128	A / C	[]	[]	[]	–
	FineWeb-Edu, 128	A / C	[]	[]	[]	–
	wiki2, 32	A / C	[]	[]	[]	–
	wiki2, 512	A / C	[]	[]	[]	–
Solver	TurboBoA (default)	E3 / A / C	[]	[]	[]	–
	GPTQ ($r_i=1$, no dX)	E3 / A / C	[]	[]	[]	–
	GPTAQ	E3 / A / C	[]	[]	[]	–
Row weight	$r_i=1$ in gain only	C	[]	[]	[]	–
Free-element filter	Fisher on non-free only	A'	[]	[]	[]	–
Coder	Shannon	A / C	[3.008]/[3.031]	–	–	–
	Huffman	A / C	[3.071]/[3.086]	–	–	–
	tANS	A / C	[]	–	–	–